

Documento Técnicas e Casos de Teste

Projeto: TOT — Pinhalense Auxílio de Pesquisa e Automatização

Tecnologia: Flutter (Dart) + Firebase + Groq API (Llama 4 Scout)

Arquitetura: Stateful Widgets + Services + Backend Firebase

Norma aplicada: ISO/IEC/IEEE 29119-4

1. Técnicas Utilizadas

Técnica	Finalidade
Particionamento de Equivalência	Separar entradas válidas e inválidas (e-mail, senha, índice)
Valor Limite	Validar campos vazios e limites do payload (3 imagens)
Transição de Estado	Validar mudança de estado no upload e na resposta da IA
Teste Baseado em Cenário	Validar fluxo completo entre telas e com a API
Tabela de Decisão	Tratamento dos retornos do laService (erro, índice válido, índice 0)

2. Derivação das Condições de Teste

CT01 — Validar saneamento de entrada com tag HTML

Técnica Utilizada:

- Particionamento de Equivalência

Justificativa:

Entradas contendo tags HTML representam classe inválida e devem ter as tags removidas.

TC01 — `sanitize()` remove tags HTML

Entradas:

input = "<script>alert(1)</script>Olá"

Resultado Esperado:

Retorno = "alert(1)Olá"

(tag <script> removida; mantém o texto interior)

CT02 — Validar saneamento de entrada com caracteres especiais

Técnica Utilizada:

- Particionamento de Equivalência

Justificativa:

Caracteres { } ; \ < > são vetores comuns de injeção e devem ser removidos.

TC02 — `sanitize()` remove caracteres especiais

Entradas:

input = "a;b{c}d\\e<f>g"

Resultado Esperado:

Retorno = "abcdefg"

CT03 — Validar saneamento de entrada vazia

Técnica Utilizada:

- Valor Limite

Justificativa:

String vazia é o limite inferior aceitável; o método deve retornar a própria entrada sem erro.

TC03 — sanitize() com entrada vazia

Entradas:

input = ""

Resultado Esperado:

Retorno = "" (sem exceção)

CT04 — Validar login com campos vazios

Técnica Utilizada:

- Particionamento de Equivalência
- Valor Limite

Justificativa:

Campos vazios são limite inferior inválido para o formulário.

TC04 — Login com campos vazios

Entradas:

email = "", senha = ""

Resultado Esperado:

Validador exibe mensagem "Preencha todos os campos."

Navegação NÃO ocorre.

CT05 — Validar atalho de login admin (admin/123)

Técnica Utilizada:

- Particionamento de Equivalência

Justificativa:

Existe partição válida específica com credenciais hardcoded para administrador.

TC05 — Login como admin

Entradas:

email = "admin", senha = "123"

Resultado Esperado:

Navegação para "/home" com argumento "Administrador".

CT06 — Validar login com credenciais inválidas

Técnica Utilizada:

- Particionamento de Equivalência

Justificativa:

Credenciais incorretas formam classe inválida — FirebaseAuth deve lançar exceção.

TC06 — Login inválido

Pré-condição:

FakeAuth com usuário cadastrado de senha diferente.

Resultado Esperado:

Mensagem: "E-mail ou senha inválidos."

CT07 — Validar cadastro com dados válidos

Técnica Utilizada:

- Particionamento de Equivalência

Justificativa:

Existe uma classe válida de entrada (nome, e-mail bem formado e senha preenchida).

TC07 — Cadastro com dados válidos

Entradas:

nome = "Marcelo"
email = "marcelo@email.com"
senha = "123456"

Resultado Esperado:

Conta criada e usuário redirecionado para login.

CT08 — Validar cadastro com campos vazios

Técnica Utilizada:

- Particionamento de Equivalência
- Valor Limite

TC08 — Cadastro com campos vazios

Entradas:

nome = "", email = "", senha = ""

Resultado Esperado:

Mensagem: "Preencha todos os campos."

CT09 — Validar cadastro com e-mail inválido

Técnica Utilizada:

- Particionamento de Equivalência

Justificativa:

E-mails sem @ ou sem . representam classe inválida.

TC09 — Cadastro com e-mail inválido

Entradas:

email = "marceloemail.com"

Resultado Esperado:

Mensagem: "Informe um e-mail válido."

CT10 — Validar adição de imagem ao banco

Técnica Utilizada:

- Transição de Estado

Justificativa:

Estado do banco muda de "N imagens" para "N+1 imagens" após upload.

TC10 — Adicionar imagem

Pré-condição:

FakeFirestoreStorage e FakeFirestore vazios.

Resultado Esperado:

Coleção banco_imagens contém 1 documento com imageUrl, storagePath, nome e adicionadoEm.

CT11 — Validar remoção de imagem do banco

Técnica Utilizada:

- Transição de Estado

TC11 — Remover imagem

Pré-condição:

Banco com 1 documento contendo storagePath válido.

Resultado Esperado:

Documento removido do Firestore e arquivo removido do Storage.

Coleção banco_imagens fica vazia.

CT12 — Validar laService sem GROQ_API_KEY

Técnica Utilizada:

- Tabela de Decisão

Justificativa:

Decisão direta da função: se a chave estiver vazia, retorna erro sem chamar a API.

TC12 — analisarComContexto() sem chave

Entradas:

GROQ_API_KEY = ""

imagemUsuario = bytes válidos

imagensBanco = lista com 1 item

Resultado Esperado:

Retorno contém chave "erro" mencionando a falta da chave de API.

MockClient NÃO recebe nenhuma requisição.

CT13 — Validar laService com banco de imagens vazio

Técnica Utilizada:

- Valor Limite
- Tabela de Decisão

Justificativa:

Limite inferior do banco (0 imagens) deve curto-circuitar antes da chamada HTTP.

TC13 — analisarComContexto() com banco vazio

Entradas:

imagensBanco = []

Resultado Esperado:

Retorno contém chave "erro" mencionando banco vazio.

CT14 — Validar laService com retorno válido (índice 1..N)

Técnica Utilizada:

- Teste Baseado em Cenário
- Tabela de Decisão

Cenário:

Usuário envia croqui → API responde com índice 2 → app exibe a 2ª imagem do banco.

TC14 — Resposta válida da Groq

Pré-condição:

MockClient configurado para retornar 200 com {"indice": 2, "mensagem": "Layout B é o mais similar."}

Entradas:

imagensBanco com 3 itens (URLs A, B, C)

Resultado Esperado:

Retorno["imageUrl"] = URL do item B

Retorno["mensagem"] = "Layout B é o mais similar."

CT15 — Validar laService com retorno índice 0 (sem correspondência)

Técnica Utilizada:

- Tabela de Decisão

Justificativa:

Quando o modelo julga que nenhuma imagem é similar, retorna índice 0.

TC15 — Resposta com índice 0

Pré-condição:

MockClient configurado para retornar 200 com {"indice": 0, "mensagem": "Não encontrei correspondência..."}

Resultado Esperado:

Retorno["imageUrl"] = null

Retorno["mensagem"] = "Não encontrei correspondência..."

CT16 — Validar limite de 3 imagens enviadas ao modelo

Técnica Utilizada:

- Valor Limite

Justificativa:

O método deve enviar no máximo 3 imagens do banco mesmo que existam mais.

TC16 — Payload limitado a 3 imagens

Pré-condição:

imagensBanco com 5 itens.

Resultado Esperado:

Corpo da requisição enviada ao MockClient contém exatamente 3 blocos image_url do banco (além do bloco da imagem do usuário).

CT17 — Validar envio de mensagem só de texto (sem imagem)

Técnica Utilizada:

- Teste Baseado em Cenário

Cenário:

Usuário digita texto e envia sem anexar imagem → sistema instrui a anexar.

TC17 — Envio só de texto

Entradas:

text = "qual layout devo usar?"

selectedImage = null

Resultado Esperado:

Última mensagem do chat: "Para ativar a busca preditiva, anexe uma imagem usando o ícone de clipe."
laService NÃO é chamado.

3. Tabela Consolidada de Técnicas

Condição	Técnica
CT01	Particionamento
CT02	Particionamento
CT03	Valor Limite
CT04	Particionamento + Valor Limite
CT05	Particionamento
CT06	Particionamento
CT07	Particionamento
CT08	Particionamento + Valor Limite
CT09	Particionamento
CT10	Transição de Estado
CT11	Transição de Estado
CT12	Tabela de Decisão
CT13	Valor Limite + Tabela de Decisão
CT14	Cenário + Tabela de Decisão
CT15	Tabela de Decisão
CT16	Valor Limite
CT17	Cenário

4. Tabela Consolidada de Casos de Teste

ID	Caso
TC01	sanitize() remove tags HTML
TC02	sanitize() remove caracteres especiais
TC03	sanitize() com entrada vazia
TC04	Login com campos vazios
TC05	Login como admin (atalho)
TC06	Login inválido
TC07	Cadastro com dados válidos
TC08	Cadastro com campos vazios
TC09	Cadastro com e-mail inválido
TC10	Adicionar imagem ao banco
TC11	Remover imagem do banco
TC12	laService sem GROQ_API_KEY
TC13	laService com banco vazio
TC14	laService com índice válido (1..N)
TC15	laService com índice 0
TC16	Payload limitado a 3 imagens
TC17	Envio só de texto (sem imagem)

5. Conclusão da Etapa

As condições de teste identificadas no Documento A foram derivadas em casos de teste completos utilizando técnicas formais definidas pela ISO/IEC/IEEE 29119-4. Os casos cobrem o saneamento de entradas, os fluxos de autenticação, a gestão do banco de imagens e — com especial atenção — todos os caminhos de decisão do IaService (sem chave, banco vazio, índice válido, índice 0 e limite de 3 imagens). Os casos estão preparados para implementação automatizada no projeto Flutter, utilizando testes de unidade (mocktail + MockClient) e testes de integração (firebase_auth_mock + fake_cloud_firestore).